Experiment No: 08

Aim

To develop an elementary chatbot for customer interaction using predefined rules and pattern matching techniques in Python.

Software Requirements

- Python 3.x
- NLTK Library
- Tkinter (GUI Library)

Theory

What is a Chatbot?

A chatbot is a computer program designed to simulate human conversation through text or voice interactions. It acts as a virtual assistant that helps users obtain information, solve problems, or perform specific tasks.

Chatbots can be categorized into:

1. **Rule-Based Chatbots**
    - Work on predefined rules and patterns.
    - Provide responses based on matching keywords or phrases.
    - Simple to design and implement.
2. **AI-Based Chatbots**
    - Use Artificial Intelligence and Machine Learning.
    - Understand user intent and context.
    - Learn from interactions and improve over time.

Need for Chatbots

- Provide instant customer support.
- Reduce human workload.
- Available 24×7.
- Improve customer engagement.
- Handle multiple users simultaneously.

Applications of Chatbots

- Customer Support Systems
- Banking Services
- Healthcare Assistance
- E-commerce Platforms
- Educational Systems
- Travel and Hospitality Services

Algorithm

1. Import required libraries.
2. Define pattern-response pairs.
3. Create a chatbot object using NLTK's Chat module.
4. Design a graphical user interface using Tkinter.
5. Accept user input.
6. Match the input with predefined patterns.
7. Generate and display the corresponding response.
8. Continue the conversation until the user closes the application.

Program

```
import nltk
from nltk.chat.util import Chat, reflections
import tkinter as tk
from tkinter import scrolledtext

# Define chatbot responses
pairs = [
    [r"hi|hello|hey",
     ["Hello! How can I assist you today?",
      "Hi there! How can I help?"]],

    [r"how are you?",
     ["I'm just a bot, but I'm doing fine! How about you?",
      "I'm always good! How can I assist?"]],

    [r"(.*) your name?",
     ["I'm a chatbot, here to assist you."]],

    [r"bye|goodbye",
     ["Goodbye! Have a great day!",
      "Bye! Take care!"]],

    [r"(.*)",
     ["I'm not sure how to respond to that. Could you rephrase?"]]
]

# Create chatbot
```

```
chatbot = Chat(pairs, reflections)

# Function to send messages
def send_message():
    user_input = user_entry.get()

    chat_history.insert(tk.END,
                        f"You: {user_input}\n")

    response = chatbot.respond(user_input)

    chat_history.insert(tk.END,
                        f"Bot: {response}\n\n")

    user_entry.delete(0, tk.END)

# GUI setup
root = tk.Tk()
root.title("Simple Chatbot")

chat_history = scrolledtext.ScrolledText(
    root,
    wrap=tk.WORD,
    width=50,
    height=15
)
chat_history.pack(padx=10, pady=10)

user_entry = tk.Entry(root, width=40)
user_entry.pack(padx=10, pady=5)

send_button = tk.Button(
    root,
    text="Send",
    command=send_message
)
send_button.pack(pady=5)

root.mainloop()
```

Sample Output

```
You: Hello
Bot: Hello! How can I assist you today?

You: What is your name?
Bot: I'm a chatbot, here to assist you.

You: How are you?
Bot: I'm always good! How can I assist?

You: Bye
Bot: Goodbye! Have a great day!
```

Advantages

- Easy to implement.
- Provides quick responses.
- Available 24×7.
- Handles multiple customer queries.
- Reduces operational costs.

Applications

- Online Customer Support
- Banking Assistants
- E-commerce Help Desks
- Hospital Information Systems
- Educational Assistance Systems

Result

An elementary chatbot for customer interaction was successfully developed using Python, NLTK, and Tkinter. The chatbot responds to user queries through predefined rules and pattern matching techniques.